What we are Basically Doing is To check if MobileNetV2 can classify it before we modify it for skin diseases.

STEP 1:

Check if the GPU is available

```
import tensorflow as tf
```

```
print("GPU Available:", tf.config.list_physical_devices("GPU"))
```

```
GPU Available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

STEP 2:

Load Pre-trained Model (MobileNetV2) - Lightweight and accurate

1. TensorFlow → ML framework for training & running models.

2. MobileNetV2 → A lightweight pre-trained model for image classification.

3. image → Helps load and preprocess images.

4. preprocess_input → Prepares images for MobileNetV2.

5. decode_predictions → Converts model output into readable class labels.

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions
import numpy as np
import matplotlib.pyplot as plt
```

```
#loab the pre-trained model
model = MobileNetV2(weights='imagenet')
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_orde
14536120/14536120 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
```

STEP 3: Colab does not allow direct file uploads in a running session, so we use this method:

```
from google.colab import files
uploaded = files.upload()  # Manually upload an image
```

Choose Files GettyImage…f2a31d2.jpg

- **GettyImages-177384156_0-2000-7d742889400d4f0ba5233f297f2a31d2.jpg**(image/jpeg) - 100167 bytes, last modified: 3/18/2025 - 100% done

```
Saving GettyImages-177384156_0-2000-7d742889400d4f0ba5233f297f2a31d2.jpg to GettyImages-177384156_0-2000-7d742889400d4f0ba5233f2
```

img_to_array(img) → Converts the image into a numerical array.

expand_dims(img_array, axis=0) → Adds a batch dimension (since ML models expect multiple images).

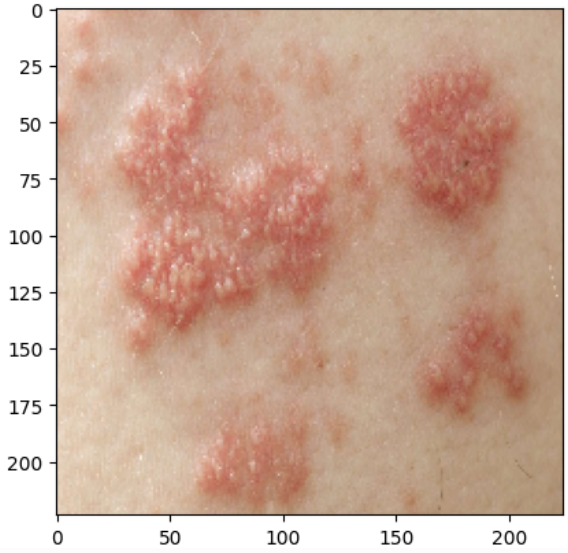
preprocess_input(img_array) → Normalizes pixel values to match what MobileNetV2 was trained on.

```
#now load and preprocess the image:
img_path = list(uploaded.keys())[0]
img = image.load_img(img_path, target_size=(224,224))#standard
plt.imshow(img)#show the image

#convert to array and preprocess
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis = 0)
image_array = preprocess_input(img_array)

#predict
predictions = model.predict(img_array)
decoded_predictions = decode_predictions(predictions, top = 3)[0]
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
  print(f"{i+1}. {label} ({score*100:.2f})")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 5s 5s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 ━━━━━━━━━━━━━━━━━━━━ 0s 1us/step
1. ice_lolly (46.20)
2. paper_towel (10.72)
3. strawberry (7.10)
```



Python

predictions = model.predict(img_array)

✅ Making a prediction

The model analyzes the image and returns a list of possible categories.

python

decoded_predictions = decode_predictions(predictions, top=3)[0] ✅ Decoding predictions

decode_predictions(predictions, top=3) → Converts model output into human-readable labels (e.g., "Dog", "Cat", etc.). [0] → Takes only the top 3 predictions for the first image.

python

for label, name, confidence in decoded_predictions: print(f"{name}: {confidence*100:.2f}%")

✅ Displaying the results

Loops through the top 3 predictions and prints the label and confidence score (probability).

STEP 4:

Fine tune for the Skin Disease Detection

```
#first upload the dataset
!pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.11/dist-packages (1.6.17)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.11/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: certifi>=2023.7.22 in /usr/local/lib/python3.11/dist-packages (from kaggle) (2025.1.31)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.11/dist-packages (from kaggle) (2.8.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from kaggle) (2.32.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.11/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.11/dist-packages (from kaggle) (2.3.0)
Requirement already satisfied: bleach in /usr/local/lib/python3.11/dist-packages (from kaggle) (6.2.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.11/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.11/dist-packages (from python-slugify->kaggle) (1.3
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->kaggle) (3.4.
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->kaggle) (3.10)
```

```
!kaggle datasets download -d shonenkov/isic2019 --unzip
```

```
403 - Forbidden - Permission 'datasets.get' was denied
```

```
#since permission get denied we have to write out api key to get the permission
files.upload()
```

Choose Files kaggle (1).json

- **kaggle (1).json**(application/json) - 76 bytes, last modified: 3/18/2025 - 100% done

```
Saving kaggle (1).json to kaggle (1).json
{'kaggle (1).json': b'{"username":"anushthasinghkushwah","key":"72e4fd622bc71a8e45eff4be111f228"}'}
```

```
#move kaggle to its correct location
!mkdir -p ~/.kaggle
!cp "kaggle (1).json" ~/.kaggle/kaggle.json
!chmod 600 ~/.kaggle/kaggle.json
```

```
#!kaggle datasets download -d shonenkov/isic2019 --unzip
```

```
import kagglehub

# # Download latest version
path = kagglehub.dataset_download("andrewmvd/isic-2019")

print("Path to dataset files:", path)
```

```
Downloading from https://www.kaggle.com/api/v1/datasets/download/andrewmvd/isic-2019?dataset_version_number=1...
100%|██████████| 9.10G/9.10G [06:55<00:00, 23.5MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1
```

Fine-tune MobileNetV2 using skin disease labels.

```
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.models import Model

x = model.output
x = Flatten()(x)#Flatten() converts it into a 1D array for input into dense layers.
x = Dense(128, activation = 'relu')(x)
x = Dense(9, activation = 'softmax')(x)#9 = no. of skin disease calsses
#Softmax activation is used for multi-class classification (outputs probabilities).

#create new model
skin_model = Model(inputs = model.input, outputs = x)#Defines a new model with original MobileNetV2 input but custom output layers.
skin_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics =['accuracy'])
#optimizer: "adam" (efficient for deep learning).
#Loss Function: "categorical_crossentropy" (used for multi-class classification).
#Metrics: "accuracy" (to measure performance).
```

1. Set up Google Colab and checked GPU availability.
2. Loaded a pre-trained MobileNetV2 model with ImageNet weights.
3. Uploaded and tested an image for predictions.
4. Fine-tuned MobileNetV2 by adding custom layers for skin disease classification.

Step 5 of downloading the dataset is already done above

STEP 6:

Preprocess the Dataset

```
#check extracted file
import os
print("Dataset files:", os.listdir(path)) #list all the files in the dataset
```

```
Dataset files: ['ISIC_2019_Training_Input', 'ISIC_2019_Training_Metadata.csv', 'ISIC_2019_Training_GroundTruth.csv']
```

```
import pandas as pd
labels_df = pd.read_csv(f"{path}/ISIC_2019_Training_GroundTruth.csv")
labels_df.head()
```

	image	MEL	NV	BCC	AK	BKL	DF	VASC	SCC	UNK
0	ISIC_0000000	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	ISIC_0000001	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	ISIC_0000002	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	ISIC_0000003	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	ISIC_0000004	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Next steps: Generate code with labels_df | View recommended plots | New interactive sheet

```
labels_df.columns
```

```
Index(['image', 'MEL', 'NV', 'BCC', 'AK', 'BKL', 'DF', 'VASC', 'SCC', 'UNK'], dtype='object')
```

STEP 7: Load Image data for Training

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
import os

# Check all available files and folders
print("Dataset contents:", os.listdir(path))

# Check inside the image folder
image_folder = f"{path}/ISIC_2019_Training_Input"
print("Files inside image folder:", os.listdir(image_folder)[:10])  # Show first 10 files
```

```
Dataset contents: ['ISIC_2019_Training_Input', 'ISIC_2019_Training_Metadata.csv', 'ISIC_2019_Training_GroundTruth.csv']
Files inside image folder: ['ISIC_2019_Training_Input']
```

```
import os

dataset_path = f"{path}/ISIC_2019_Training_Input"  # Define correct path

# Check if the path exists
if os.path.exists(dataset_path):
    print("✅ Path exists! Listing files:")
    print(os.listdir(dataset_path)[:10])  # Show first 10 files
else:
    print("❌ Path does not exist! Check dataset location.")
```

```
✅ Path exists! Listing files:
['ISIC_2019_Training_Input']
```

```
nested_folder = f"{path}/ISIC_2019_Training_Input"
print("Files inside nested folder:", os.listdir(nested_folder)[:10])  # Show first 10 files
```

```
Files inside nested folder: ['ISIC_2019_Training_Input']
```

```
import os

image_folder =  f"{path}/ISIC_2019_Training_Input" # Path to images
print("Total images:", len(os.listdir(image_folder)))
```

```
Total images: 1
```

⏭ **Step 7.2: Preprocess Images & Labels**

1. Resize images → (224x224 for MobileNetV2)
2. Normalize pixel values → (Scale between 0-1)
3. Convert labels → (One-hot encoding)

```
import os
import cv2
import numpy as np

IMG_SIZE = 224  # Adjust based on your model's input size
image_folder = f"{path}/ISIC_2019_Training_Input"  # Adjust if needed

# Function to load and preprocess images
def load_image(image_id):
    img_path = os.path.join(image_folder, image_id + ".jpg")
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))  # Resize
    img = img / 255.0  # Normalize
    return img
# Function to load and preprocess images
#os.path.join(image_folder, image_id + ".jpg") → Finds the image file path.
#cv2.imread(img_path) → Reads the image.
#cv2.cvtColor(img, cv2.COLOR_BGR2RGB) → Converts from BGR to RGB (since OpenCV loads images in BGR format).
# Resize : cv2.resize(img, (IMG_SIZE, IMG_SIZE)) → Resizes the image to 224×224 (required for MobileNetV2).
# Normalize img / 255.0 → Normalizes pixel values from 0-255 to 0-1 (helps with faster training).
# Returns the processed image.
```

⏭️ Step 7.3: Convert Labels & Create Training Data 🔹 What We'll Do:

1️⃣ Extract image IDs and their corresponding disease labels from ISIC_2019_Training_GroundTruth.csv.

2️⃣ One-hot encode labels → Convert them into a machine-readable format.

3️⃣ Create training data (images + labels).

```
import os
print(os.path.exists(path))  # Should return True
```

```
True
```

```
print(image_folder)  # Should point to the folder with images
```

```
/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/ISIC_2019_Training_Input
```

```
import os
print(os.listdir(image_folder)[:10])  # See first 10 images
```

```
['ISIC_2019_Training_Input']
```

```
def load_image(image_id):
    img_path = os.path.join(image_folder, image_id + ".jpg")
    if not os.path.exists(img_path):
        print(f"Image not found: {img_path}")
        return np.zeros((IMG_SIZE, IMG_SIZE, 3))  # Return blank image

    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img = img / 255.0
    return img
```

```
path = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1"  # Adjust based on your system

import pandas as pd
labels_df = pd.read_csv(f"{path}/ISIC_2019_Training_GroundTruth.csv")
labels_df = labels_df.sample(5000, random_state=42)  # Reduce dataset size
```

```
import os

image_folder = f"{path}/ISIC_2019_Training_Input"
available_images = os.listdir(image_folder)[:10]  # Show first 10 images
print("Sample images available:", available_images)
```

```
Sample images available: ['ISIC_2019_Training_Input']
```

found that some images where missing

```
missing_images = ["ISIC_0010070", "ISIC_0029231", "ISIC_0033173", "ISIC_0072375"]

for img in missing_images:
    if img in labels_df["image"].values:
        print(f"{img} exists in CSV but is missing in the folder.")
    else:
        print(f"{img} does not exist in CSV.")
```

```
ISIC_0010070 exists in CSV but is missing in the folder.
ISIC_0029231 exists in CSV but is missing in the folder.
ISIC_0033173 exists in CSV but is missing in the folder.
ISIC_0072375 exists in CSV but is missing in the folder.
```

```
import os

dataset_path = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/"

# Recursively find all image files
all_images = []
for root, dirs, files in os.walk(dataset_path):
    for file in files:
        if file.endswith(".jpg"):
            all_images.append(os.path.join(root, file))

print(f"Total images found: {len(all_images)}")
print("Sample image paths:", all_images[:5])
```

```
Total images found: 25331
Sample image paths: ['/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/ISIC_2019_Training_Input/ISIC_2019_Training
```

Move images to correct folder

```
import os
import shutil

# Define paths
wrong_folder = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/ISIC_2019_Training_Input/ISIC_2019_Training_Input/"
correct_folder = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/ISIC_2019_Training_Input/"

# Move images to the correct location
for file_name in os.listdir(wrong_folder):
    src_path = os.path.join(wrong_folder, file_name)
    dst_path = os.path.join(correct_folder, file_name)
    shutil.move(src_path, dst_path)

# Verify fix
print("✅ All images moved to the correct folder!")

# Remove the extra empty folder
os.rmdir(wrong_folder)
print("✅ Extra folder removed!")
```

```
✅ All images moved to the correct folder!
✅ Extra folder removed!
```

```
print("✅ Total images now in the correct folder:", len(os.listdir(correct_folder)))
```

```
✅ Total images now in the correct folder: 25333
```

Double-click (or enter) to edit

```
missing_images = ["ISIC_0010070", "ISIC_0029231", "ISIC_0033173", "ISIC_0072375"]

for img in missing_images:
    if img in labels_df["image"].values:
        print(f"{img} exists in CSV but is missing in the folder.")
    else:
        print(f"{img} does not exist in CSV.")
```

```
ISIC_0010070 exists in CSV but is missing in the folder.
ISIC_0029231 exists in CSV but is missing in the folder.
ISIC_0033173 exists in CSV but is missing in the folder.
ISIC_0072375 exists in CSV but is missing in the folder.
```

Re-download Dataset (Only If Images Are Missing) Since the dataset is large, avoid redownloading the entire thing. Instead, try:

```
#!kaggle datasets download -d andrewmvd/isic-2019 --unzip


import os
import cv2
import numpy as np

# Corrected image folder path
image_folder = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/ISIC_2019_Training_Input/"

# Function to load and preprocess images
def load_image(image_id):
    img_path = os.path.join(image_folder, image_id + ".jpg")  # Correct path
    if not os.path.exists(img_path):  # Check if image exists
        print(f"⚠️ Image not found: {img_path}")
        return None  # Return None for missing images

    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))  # Resize
    img = img / 255.0  # Normalize
    return img

# Load first 5 images to check
sample_images = [load_image(img_id) for img_id in labels_df["image"][:5] if load_image(img_id) is not None]
print("✅ Sample image shape:", sample_images[0].shape if sample_images else "No valid images found")
```

```
✅ Sample image shape: (224, 224, 3)
```

```
import os

dataset_path = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/"

print("Dataset contents:", os.listdir(dataset_path))
print("Images folder contents:", os.listdir(os.path.join(dataset_path, "ISIC_2019_Training_Input")))
```

```
Dataset contents: ['ISIC_2019_Training_Input', 'ISIC_2019_Training_Metadata.csv', 'ISIC_2019_Training_GroundTruth.csv']
Images folder contents: ['ISIC_0054909.jpg', 'ISIC_0010481.jpg', 'ISIC_0014855_downsampled.jpg', 'ISIC_0014762_downsampled.jpg',
```

```
image_folder = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/ISIC_2019_Training_Input"
image_files = os.listdir(image_folder)
print(f"Total images found: {len(image_files)}")
```

```
Total images found: 25333
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import pandas as pd
import os

# Define constants
IMG_SIZE = 128  # Reduce size to save memory (originally 256+)
BATCH_SIZE = 16  # Reduce batch size to avoid crashes

# Load the labels CSV
labels_df = pd.read_csv(f"{path}/ISIC_2019_Training_GroundTruth.csv")

# Add '.jpg' extension to image IDs (if missing)
labels_df["image"] = labels_df["image"].astype(str) + ".jpg"

# Define the correct image directory
image_folder = f"{path}/ISIC_2019_Training_Input"

# Data Generator with normalization
train_datagen = ImageDataGenerator(
    rescale=1./255,  # Normalize images
    validation_split=0.2  # 80% training, 20% validation
```

```
)

# Training Data Generator
train_generator = train_datagen.flow_from_dataframe(
    dataframe=labels_df,
    directory=image_folder,
    x_col="image",  # Column containing image filenames
    y_col=labels_df.columns[1:],  # Disease labels (multi-label classification)
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="raw",  # Use "categorical" for one-hot encoding
    subset="training"
)

# Validation Data Generator
val_generator = train_datagen.flow_from_dataframe(
    dataframe=labels_df,
    directory=image_folder,
    x_col="image",
    y_col=labels_df.columns[1:],
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="raw",
    subset="validation"
)

print("✅ Data generators are set up correctly!")
```

```
Found 20265 validated image filenames.
Found 5066 validated image filenames.
✅ Data generators are set up correctly!
```

STEP7.4

Model Training

Now, we'll: ✅ Normalize the images (already done in load_image()).

✅ Define a Convolutional Neural Network (CNN) for skin disease classification.

✅ Compile & train the model.

Define CNN Model

```
label_columns = labels_df.columns[1:]
```

```
import tensorflow as tf
from tensorflow.keras import layers, models

IMG_SIZE = 128  # Make sure it's defined before using it

# Define the CNN model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_SIZE, IMG_SIZE, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),  # Helps prevent overfitting
    layers.Dense(len(label_columns), activation='softmax')  # Output layer with multiple classes
])

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

print("✅ Model Defined Successfully!")
```

```
✅ Model Defined Successfully!
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Train the Model

```
import tensorflow as tf
print(tf.__version__)  # Should return a version like 2.x.x
```

```
2.18.0
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data Augmentation & Normalization
train_datagen = ImageDataGenerator(
    rescale=1./255,  # Normalize pixel values
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.2  # 80% training, 20% validation
)

# Directory Paths
train_dir = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/ISIC_2019_Training_Input"

# Load images in batches
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=32,  # Reduce if RAM crashes again
    class_mode="categorical",
    subset="training"
)

val_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=32,
    class_mode="categorical",
    subset="validation"
)

print("✅ Data Generators Ready!")
```

```
Found 0 images belonging to 0 classes.
Found 0 images belonging to 0 classes.
✅ Data Generators Ready!
```

NO class so we gonna create a classwise folder

```
import os

dataset_path = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/"
print(os.listdir(dataset_path))
```

```
['ISIC_2019_Training_Input', 'ISIC_2019_Training_Metadata.csv', 'ISIC_2019_Training_GroundTruth.csv']
```

```
import os
import shutil
import pandas as pd

# Paths
csv_path = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/ISIC_2019_Training_GroundTruth.csv"

image_folder = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/versions/1/ISIC_2019_Training_Input"
output_folder = "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/organized_dataset"

# Load CSV
df = pd.read_csv(csv_path)
```

```
# Class names (excluding 'image' column)
class_names = df.columns[1:]

# Create class-wise folders
for class_name in class_names:
    os.makedirs(os.path.join(output_folder, class_name), exist_ok=True)

# Move images to respective folders
for _, row in df.iterrows():
    image_id = row['image']
    class_label = class_names[row[1:].values.argmax()]  # Get the class with highest probability

    src = os.path.join(image_folder, f"{image_id}.jpg")
    dest = os.path.join(output_folder, class_label, f"{image_id}.jpg")

    if os.path.exists(src):
        shutil.move(src, dest)

print("✅ Images Organized Successfully!")
```

```
✅ Images Organized Successfully!
```

```
train_generator = train_datagen.flow_from_directory(
    "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/organized_dataset",
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=32,
    class_mode="categorical",
    subset="training"
)

val_generator = train_datagen.flow_from_directory(
    "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/organized_dataset",
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=32,
    class_mode="categorical",
    subset="validation"
)
```

```
Found 20269 images belonging to 9 classes.
Found 5062 images belonging to 9 classes.
```

1. Add validation_split in ImageDataGenerator Modify your train_datagen like this:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

IMG_SIZE = 128
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2  # Ensure this is set
)

train_generator = train_datagen.flow_from_directory(
    "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/organized_dataset",
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    subset="training"  # Use training subset
)

val_generator = train_datagen.flow_from_directory(
    "/root/.cache/kagglehub/datasets/andrewmvd/isic-2019/organized_dataset",
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    subset="validation"  # Use validation subset
)
```

```
Found 20269 images belonging to 9 classes.
Found 5062 images belonging to 9 classes.
```

2. Check If Images Are Being Loaded After defining train_generator and val_generator, run this to check:

```
print(f"Training Samples: {train_generator.samples}")
print(f"Validation Samples: {val_generator.samples}")
```

```
Training Samples: 20269
Validation Samples: 5062
```

Double-click (or enter) to edit

```
EPOCHS = 10  # Adjust based on available resources

history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=EPOCHS
)
```

```
Epoch 1/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 310s 479ms/step - accuracy: 0.5188 - loss: 1.3743 - val_accuracy: 0.4919 - val_loss: 1.3047
Epoch 2/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 300s 473ms/step - accuracy: 0.5850 - loss: 1.1824 - val_accuracy: 0.5314 - val_loss: 1.2781
Epoch 3/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 332s 524ms/step - accuracy: 0.6086 - loss: 1.1030 - val_accuracy: 0.5198 - val_loss: 1.2745
Epoch 4/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 294s 464ms/step - accuracy: 0.6183 - loss: 1.0822 - val_accuracy: 0.5302 - val_loss: 1.2708
Epoch 5/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 290s 457ms/step - accuracy: 0.6313 - loss: 1.0275 - val_accuracy: 0.5478 - val_loss: 1.2765
Epoch 6/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 289s 455ms/step - accuracy: 0.6522 - loss: 0.9757 - val_accuracy: 0.5251 - val_loss: 1.3179
Epoch 7/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 294s 463ms/step - accuracy: 0.6585 - loss: 0.9422 - val_accuracy: 0.5253 - val_loss: 1.3226
Epoch 8/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 318s 457ms/step - accuracy: 0.6814 - loss: 0.8845 - val_accuracy: 0.5490 - val_loss: 1.2693
Epoch 9/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 290s 458ms/step - accuracy: 0.6950 - loss: 0.8390 - val_accuracy: 0.5448 - val_loss: 1.4629
Epoch 10/10
634/634 ━━━━━━━━━━━━━━━━━━━━ 292s 460ms/step - accuracy: 0.7066 - loss: 0.7998 - val_accuracy: 0.5225 - val_loss: 1.6257
```

Evaluate the Model

```
train_loss, train_acc = model.evaluate(train_generator)
val_loss, val_acc = model.evaluate(val_generator)

print(f"✅ Training Accuracy: {train_acc:.4f}, Training Loss: {train_loss:.4f}")
print(f"✅ Validation Accuracy: {val_acc:.4f}, Validation Loss: {val_loss:.4f}")
```

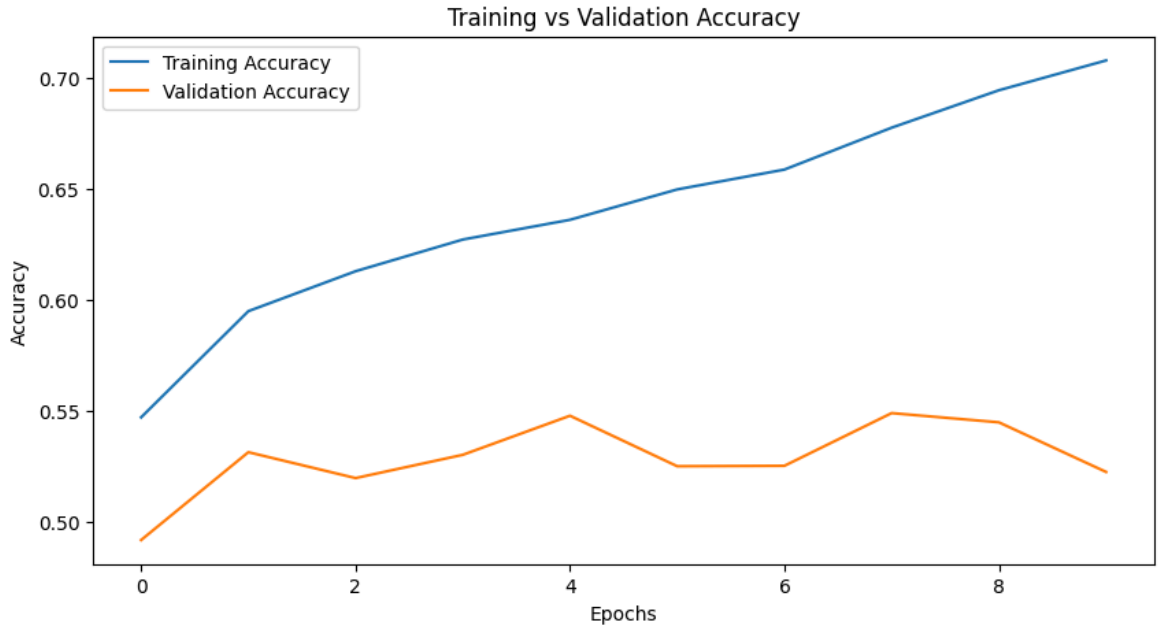
```
634/634 ━━━━━━━━━━━━━━━━━━━━ 250s 395ms/step - accuracy: 0.7595 - loss: 0.6621
159/159 ━━━━━━━━━━━━━━━━━━━━ 45s 285ms/step - accuracy: 0.5247 - loss: 1.6309
✅ Training Accuracy: 0.7607, Training Loss: 0.6543
✅ Validation Accuracy: 0.5225, Validation Loss: 1.6257
```

Visualize Training Progress

```
import matplotlib.pyplot as plt

# Plot training & validation accuracy
plt.figure(figsize=(10,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training vs Validation Accuracy')
plt.show()

# Plot training & validation loss
plt.figure(figsize=(10,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Training vs Validation Loss')
plt.show()
```



save the model

```
model.save("skin_disease_model.h5")
print("✅ Model Saved Successfully!")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
✅ Model Saved Successfully!
```

Open the below one format

```
model.save("skin_disease_model.keras")
print("✅ Model Saved in Keras Format!")
```

```
✅ Model Saved in Keras Format!
```

Make Predictions

```
from tensorflow.keras.preprocessing import image
import numpy as np

# Load a test image (change path accordingly)
img_path = "/GettyImages-177384156_0-2000-7d742889400d4f0ba5233f297f2a31d2.jpg"
```

```
img = image.load_img(img_path, target_size=(IMG_SIZE, IMG_SIZE))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)  # Convert to batch format
img_array /= 255.0  # Normalize

# Make prediction
pred = model.predict(img_array)
pred_class = np.argmax(pred)  # Get class index

# Print result
print(f"Predicted Class: {pred_class}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 824ms/step
Predicted Class: 2
```

```
print(train_generator.class_indices)
```

```
{'AK': 0, 'BCC': 1, 'BKL': 2, 'DF': 3, 'MEL': 4, 'NV': 5, 'SCC': 6, 'UNK': 7, 'VASC': 8}
```